```
function Nps =
ps_densification(Btemp,Bdop,grid_array_az,grid_array_r,Nifgs,Npsc,Npsc_selection
s,Npsp,Nref,ps_model,final_model,final_althyp_index,std_param,sig2_est,breakpoin
t,breakpoint2,Namp_disp_bins,Ndens_iterations,densification_flag,filenames_outpu
t,filenames_ifgs,filenames_h2ph,detrend_flag,defo_model_flag,ps_aoi,dens_method,
dens_check,filename_water_mask,crop,crop_in,Nest,master_index,Nslc)

% ps_densification
%
% Input:    - Btemp               temporal baselines [year]
%           - Bdop                Doppler baselines [Hz]
%           - grid_array_az       grid points azimuth direction
%           - grid_array_r        grid points range direction
%           - Nifgs               number of interferograms
%           - Npsc                number of psc's
%           - Npsc_selections     number of psc selections
%           - Npsp                number of psp's
%           - Nref                number of reference points
%           - ps_model            models used for wrapped data
%           - final_model         model used for unwrapped data
%           - final_althyp_index  alternative hypothesis of model
%           - std_param           standard deviations for pseudo-
%                                 observations
%           - sig2_est            estimated variance components
%           - breakpoint          breakpoint in case of double
%                                 linear model
%           - breakpoint2         second breakpoint, should be larger
%                                 than first breakpoint
%           - Namp_disp_bins      number of bins for amplitude
%                                 dispersion (ps_eval_method =
%                                 'whole' only)
%           - Ndens_iterations    number of densification iterations
%                                 (ps_eval_method = 'whole' only)
%           - densification_flag  region growing densification
%                                 flag, 'yes' or 'no'
%           - filenames_output    filenames for output
%           - filenames_ifgs      filenames of interferograms
%           - filenames_h2ph      filenames of height-to-phase
%                                 values
%           - detrend_flag        detrend flag, 'yes' or 'no'
%           - defo_model_flag     deformation model flag, 'yes or
%                                 'no'
%           - ps_aoi              area of interest, 'filename',
%                                 [az0 azN r0 rN] or []
%           - dens_method         densification method, 'orig' or
%                                 'adapt'
%           - dens_check          densification check, 'nocheck',
%                                 '2of3' or '3of3'
%           - filename_water_mask mask for water areas, 'filename' or []
%           - crop                borders of final crop
%           - crop_in             borders of input crops
%           - Nest                number of connecting arcs to estimate
%           - master_index        index of master acquisition
%           - Nslc                number of slc's
%
% Output:   - Nps                 number of ps's
%
% ------------------------------------------------------------------------
% File............: ps_densification.m
% Version & Date..: 1.7.2.16, 12-DEC-2009
% Authors.........: Freek van Leijen
%                   Gini Ketelaar
%                   Delft Institute of Earth Observation and Space Systems
%                   Delft University of Technology
```

```
% ----------------------------------------------------------------------
%
% This software is developed by Delft University of Technology and is
% intended for scientific use only. Applications for commercial use are
% prohibited.
%
% Copyright (c) 2004-2009 Delft University of Technology, The Netherlands
%
% Change log
% v1.7.2.12, Freek van Leijen
% - filenames_output in cells
% v1.7.2.17, Freek van Leijen
% - mode_fix option for densification check
% - any number of connecting arcs (together with mode_fix)
% v1.7.5.3, Sami Samiei Esfahany
% - bug fixed, line 539: psp_acheck_grid(ps_index(ps_index2),:,:) = repmat(NaN,
[length(ps_index<<<2>>>) Nifgs 3]);
%


% ----------------------------------------------------------------------
% Initialize
% ----------------------------------------------------------------------

ps_set_globals;


Nps = zeros(Npsc_selections,1);
step1_orig = 1; % [m]
step2_orig = 0.001; % [m], = 1 mm

if final_model<2
  error('The final model should be larger than ''1''.');
end


% force infinite threshold in case 'nocheck'
switch dens_check
  case 'nocheck'
    varfac_threshold = inf;
end

%overwrite Nest (if necessary)
switch dens_check
  case 'nocheck'
    Nest = 1;
  case {'2of3','3of3'}
    Nest = 3;
  case {'mode_fix'}
    if Nest<3
      error(['In case of ''mode_fix'', the number of arcs (Nest) ' ...
             'should be at least 3.'])
    end
  otherwise
    error(['You specified a wrong densification check, ' ...
           'should be ''nocheck'', ''2of3'', ''3of3'' or ''mode_fix''.']);
end

% ----------------------------------------------------------------------
% Construct stochastic model
% ----------------------------------------------------------------------

if isempty(sig2_est)
  if model(1) == 1 % master atmosphere stochastic
```

```
    sig2 = [(pi*15/180)^2; repmat((pi*20/180)^2,Nifgs,1)];
  else % master atmosphere estimated
    sig2 = [0; repmat((pi*30/180)^2,Nifgs,1)];
  end
else
  sig2 = sig2_est;
end

Qy = repmat(2*sig2(1),Nifgs,Nifgs);
for v = 1:Nifgs
  Qy(v,v) = Qy(v,v)+2*sig2(v+1);
end
invQy = inv(Qy);


% ------------------------------------------------------------------------
% Select desired model (for final estimation only, not unwrapping!)
% ------------------------------------------------------------------------

[Npar,par_index,covar_index] = ps_model_definitions('Npar',final_model);
[ps_annotation,par_index,covar_index] =
ps_model_definitions('annotation',final_model);
redun = Nifgs-Npar; %redundancy



% ------------------------------------------------------------------------
% Loop
% ------------------------------------------------------------------------

for z = 1:Npsc_selections


  % ----------------------------------------------------------------------
  % Read data
  % ----------------------------------------------------------------------

  psc_fid = fopen([project_id '_psc_sel' num2str(z) '.raw'],'r');
  psc_data = fread(psc_fid,[2*Nifgs+7 Npsc(z)],'double')';
  fclose(psc_fid);

  psc_array = psc_data(:,1:2);
  psc_az = psc_data(:,5);
  psc_r = psc_data(:,6);
  psc_phase = psc_data(:,7:Nifgs+6);
  psc_h2ph = psc_data(:,Nifgs+7:2*Nifgs+6);
  psc_amp_disp = psc_data(:,2*Nifgs+7);
  clear psc_data

  psc_validation_fid = fopen([project_id '_psc_validation_sel' ...
                      num2str(z) '.raw'],'r');
  psc_amp = fread(psc_validation_fid,[Nslc Npsc(z)],'double')';
  fclose(psc_validation_fid);

  psc_results_fid = fopen([project_id '_psc_results_sel' num2str(z)
'.raw'],'r');
  psc_data = fread(psc_results_fid,[3*Nifgs+Npar_max+2 Npsc(z)],'double')';
  fclose(psc_results_fid);

  psc_param = psc_data(:,1:Npar_max);
  psc_ens_coh = psc_data(:,Npar_max+1);
  psc_sig2hat = psc_data(:,Npar_max+2);
  psc_phase_res = psc_data(:,Npar_max+3:Npar_max+Nifgs+2);
  psc_acheck = psc_data(:,Npar_max+Nifgs+3:Npar_max+2*Nifgs+2);
```

```
  psc_defo = psc_data(:,Npar_max+2*Nifgs+3:Npar_max+3*Nifgs+2);
  clear psc_data

  switch weighting
    case 'vce'

      psc_results_fid = fopen([project_id '_psc_results_covar_sel' num2str(z)
'.raw'],'r');
      psc_covar = fread(psc_results_fid,[Npar_max*(Npar_max+1)/2
Npsc(z)],'double')';
      fclose(psc_results_fid);

  end

  psc_atmo_fid = fopen([project_id '_psc_atmo_sel' num2str(z) '.raw'],'r');
  psc_atmo = fread(psc_atmo_fid,[Nifgs Npsc(z)],'double')';
  fclose(psc_atmo_fid);

  switch defo_model_flag
    case 'yes'
      psc_defo_fid = fopen([project_id '_psc_defo_sel' num2str(z) '.raw'],'r');
      psc_defo_model = fread(psc_defo_fid,[Nifgs Npsc(z)],'double')';
      fclose(psc_defo_fid);
  end

  ref_fid = fopen([project_id '_ref_sel' num2str(z) '.raw'],'r');
  ref_array = fread(ref_fid,[3 Nref(z)],'double')';
  fclose(ref_fid);
  ref_index = ref_array(1,1); % index of general reference point


  % ---------------------------------------------------------------------
  % Initialize
  % ---------------------------------------------------------------------

  Nps(z) = 0;

  psc = find(psc_array(:,2)~=0); % remove isolated psc
  psc_az_new = psc_az(psc);
  psc_r_new = psc_r(psc);
  psc_h2ph_new = psc_h2ph(psc,:);
  psc_phase_new = psc_phase(psc,:);
  psc_amp_new = psc_amp(psc,:);
  psc_amp_disp_new = psc_amp_disp(psc);
  psc_param_new = psc_param(psc,:);
  psc_ens_coh_new = psc_ens_coh(psc);
  psc_sig2hat_new = psc_sig2hat(psc);
  psc_phase_res_new = psc_phase_res(psc,:);
  psc_acheck_new = psc_acheck(psc,:);
  psc_defo_new = psc_defo(psc,:);
  psc_atmo_new = psc_atmo(psc,:);
  [temp_index] = find(psc<ref_index);
  ref_index = length(temp_index)+1; %update ref_index
  Nps(z) = Nps(z)+length(psc);

  psc_phase_unw = 2*pi*psc_acheck_new+psc_phase_new-
repmat(psc_phase_new(ref_index,:),Nps(z),1);

  switch defo_model_flag
    case 'yes'
      psc_defo_model_new = psc_defo_model(psc,:);
      psc_phase_unw = psc_phase_unw+psc_defo_model_new;
  end
```

```
  comment_line = ones(Nps(z),1);
  comment_line(ref_index) = 0;

  ps_fid = fopen([project_id '_ps_results_' results_id '_sel' num2str(z)
'.raw'],'w');
  fwrite(ps_fid,[comment_line psc_az_new psc_r_new psc_param_new ...
                 repmat(psc_ens_coh_new,1,2) psc_sig2hat_new ...
                       psc_phase_unw psc_defo_new psc_atmo_new]','double');

  switch weighting
    case 'vce'
      psc_covar_new = psc_covar(psc,:);
      ps_covar_fid = fopen([project_id '_ps_results_covar_' results_id ...
                            '_sel' num2str(z) '.raw'],'w');
      fwrite(ps_covar_fid,psc_covar_new','double');
  end

  switch run_mode
    case 'validation'

      %read master atmo
      psc_atmo_master_fid = fopen([project_id '_psc_atmo_master1_sel' num2str(z)
'.raw'],'r');
      psc_atmo_master = fread(psc_atmo_master_fid,[Npsc(z) 1],'double');
      fclose(psc_atmo_master_fid);

      psc_atmo_master_new = psc_atmo_master(psc);

      %create atmo per slc
      %atmo_slave = atmo_master - atmo_ifgs
      psc_atmo_slc = [psc_atmo_new(:,1:master_index-1) zeros(size(psc)) ...
                      psc_atmo_new(:,master_index:end)];
      psc_atmo_slc = repmat(psc_atmo_master_new,1,Nifgs+1)- ...
          psc_atmo_slc;

      %create ts
      psc_ts = m2ph*[psc_defo_new(:,1:master_index-1) zeros(size(psc)) ...
                       psc_defo_new(:,master_index:end)];
      %write data
      ps_valid_out = [(1:Nps(z))' psc_az_new-1 psc_r_new-1 psc_amp_disp_new ...
                   psc_amp_new psc_phase_new m2ph*psc_h2ph_new ...
                   psc_param_new(:,1) 1000*psc_param_new(:,4)
psc_ens_coh_new ...
                   psc_atmo_new psc_atmo_slc psc_ts psc_phase_res_new];

      ps_valid_out_fid = fopen([project_id '_ps_validation_' results_id ...
                            '_sel' num2str(z) '.raw'],'w');
      fwrite(ps_valid_out_fid,ps_valid_out','double');

      ps_rel_valid_out_fid = fopen([project_id '_ps_validation_rel_'
results_id ...
                            '_sel' num2str(z) '.raw'],'a');
  end


  % ------------------------------------------------------------------------
  % Calculate parameters for potential ps or the whole image
  % ------------------------------------------------------------------------

  switch ps_eval_method
    case 'psp'


      % ------------------------------------------------------------------
```

```
      % Determine buffersize
      % --------------------------------------------------------------------

      Npsp_buffer = floor(max_mem_buffer/(5*(2*Nifgs+2)*8));

      if (Npsp_buffer>=Npsp(z))
        Npsp_buffer = Npsp(z);
        Nbuffers = 1;
        Npsp_rem_buffer = 0;
      else
        Nbuffers = floor(Npsp(z)/Npsp_buffer);
        Npsp_rem_buffer = rem(Npsp(z),Npsp_buffer);
        if (Npsp_rem_buffer > 0)
          Nbuffers = Nbuffers + 1;
        end
      end

      psp_fid = fopen([project_id '_psp_sel' num2str(z) '.raw'],'r');
      psp_atmo_fid = fopen([project_id '_psp_atmo_sel' num2str(z) '.raw'],'r');
      psp_atmo_master_fid = fopen([project_id '_psp_atmo_master1_sel' num2str(z)
'.raw'],'r');

      psp_validation_fid = fopen([project_id '_psp_validation.raw'],'r');

      switch defo_model_flag
        case 'yes'
          psp_defo_fid = fopen([project_id '_psp_defo_sel' num2str(z)
'.raw'],'r');
      end

      for v = 1:Nbuffers

        if (v == Nbuffers)&&(Npsp_rem_buffer~=0)
          Npsp_buffer = Npsp_rem_buffer;
        end

        % --------------------------------------------------------------------
        % Read data
        % --------------------------------------------------------------------

        psp_data = fread(psp_fid,[2*Nifgs+4 Npsp_buffer],'double')';

        psp_grid_az = psp_data(:,1);
        psp_grid_r = psp_data(:,2);
        psp_az = psp_data(:,3);
        psp_r = psp_data(:,4);
        psp_phase = psp_data(:,5:Nifgs+4);
        psp_h2ph = psp_data(:,Nifgs+5:2*Nifgs+4);
        clear psp_data

        psp_atmo = fread(psp_atmo_fid,[Nifgs Npsp_buffer],'double')';
        psp_atmo_master = fread(psp_atmo_master_fid,[Npsp_buffer 1],'double');

        switch defo_model_flag
          case 'yes'
            psp_defo_model = fread(psp_defo_fid,[Nifgs Npsp_buffer],'double')';

        end

        psp_data = fread(psp_validation_fid,[Nslc+1 Npsp_buffer],'double')';
        psp_amp_disp = psp_data(:,1);
        psp_amp = psp_data(:,2:end);
        clear psp_data
```

```
        % ------------------------------------------------------------
        % Remove PSCs from set of PSPs
        % ------------------------------------------------------------

        [dummy,psp] = setdiff([psp_az psp_r],[psc_az_new psc_r_new],'rows');
        psp_grid_az = psp_grid_az(psp);
        psp_grid_r = psp_grid_r(psp);
        psp_az = psp_az(psp);
        psp_r = psp_r(psp);
        psp_phase = psp_phase(psp,:);
        psp_h2ph = psp_h2ph(psp,:);
        psp_atmo = psp_atmo(psp,:);
        psp_atmo_master= psp_atmo_master(psp);

        switch defo_model_flag
          case 'yes'
            psp_defo_model = psp_defo_model(psp,:);
        end

        psp_amp_disp = psp_amp_disp(psp);
        psp_amp = psp_amp(psp,:);


        % ------------------------------------------------------------
        % Calculate parameters for potential ps per gridcell
        % ------------------------------------------------------------

        index_grid = [psp_grid_az psp_grid_r];
        index_grid = unique(index_grid,'rows');
        Ngrid = size(index_grid,1);

        for g = 1:Ngrid

          % ------------------------------------------------------------
          % Output to screen
          % ------------------------------------------------------------

          if rem(g,10) == 0
            fprintf(1,'\n');
            fprintf(1,'Processing gridcell %4.0f of total %4.0f in buffer %4.0f
of total %4.0f\n',g,Ngrid,v,Nbuffers);
          end


          % ------------------------------------------------------------
          % Determine reference points for gridcell
          % ------------------------------------------------------------

          center_az = floor((grid_array_az(index_grid(g,1),1) +
grid_array_az(index_grid(g,1),2))/2);
          center_r = floor((grid_array_r(index_grid(g,2),1) +
grid_array_r(index_grid(g,2),2))/2);
          distance = sqrt(((psc_az_new-center_az)*az_spacing).^2 + ((psc_r_new-
center_r)*r_spacing).^2);
          [dummy,ref] = sort(distance);
          ref = ref(1:Nest);


          % ------------------------------------------------------------
          % Find potential ps in gridcell
          % ------------------------------------------------------------

          psp_index = find(ismember([psp_grid_az
```

```
psp_grid_r],index_grid(g,:),'rows'));

          Npsp_grid = length(psp_index);
          psp_az_grid = psp_az(psp_index);
          psp_r_grid = psp_r(psp_index);
          psp_param_grid = repmat(NaN,[Npsp_grid Npar_max Nest]);
          psp_ens_coh_local = repmat(NaN,Npsp_grid,Nest);
          psp_acheck_grid = repmat(NaN,[Npsp_grid Nifgs Nest]);
          psp_atmo_grid = psp_atmo(psp_index,:);
          psp_atmo_master_grid = psp_atmo_master(psp_index);
          psp_phase_grid = psp_phase(psp_index,:);
          psp_h2ph_grid = psp_h2ph(psp_index,:);
          psp_amp_disp_grid = psp_amp_disp(psp_index);
          psp_amp_grid = psp_amp(psp_index,:);

          switch defo_model_flag
            case 'yes'
              psp_defo_model_grid = psp_defo_model(psp_index,:);
          end

          switch dens_method
            case 'orig'
              Nmodels = 1;
            case 'adapt'
              Nmodels = length(ps_model);
            otherwise
              error('You specified a wrong densification method, should be
''orig'' or ''adapt''.');
          end

          model_index = 1;
          ps_index = 1:Npsp_grid;

          while model_index<=Nmodels

            Nps_index = length(ps_index);


            for h = 1:Nest
              psp_dh2ph_grid = (psp_h2ph(psp_index(ps_index),:) +
repmat(psc_h2ph_new(ref(h),:),Nps_index,1))./2; % take mean for arc
              psp_dphase_grid = psp_phase(psp_index(ps_index),:) -
repmat(psc_phase_new(ref(h),:),Nps_index,1);


              % ------------------------------------------------------
              % Estimate the parameters
              % ------------------------------------------------------

              switch ps_method
                case 'perio'


[psp_param_grid(ps_index,:,h),psp_acheck_grid(ps_index,:,h),psp_model_info,psp_e
ns_coh_local(ps_index,h)] =
ps_periodogram_estimation(psp_dh2ph_grid,Btemp,psp_dphase_grid,ps_model(model_in
dex),std_param,step1_orig,step2_orig);

                case {'ils','boot'}


[psp_param_grid(ps_index,:,h),psp_acheck_grid(ps_index,:,h),psp_model_info,psp_v
arfac_best,psp_ens_coh_local(ps_index,h)] =
ps_ils_and_bootstrap_estimation(psp_dh2ph_grid,Btemp,Bdop,psp_dphase_grid,ps_mod
```

```
el(model_index),std_param,sig2_est,breakpoint,breakpoint2);

              end
            end

            switch dens_check
              case {'2of3','3of3','mode_fix'}

                xcheck = repmat(NaN,[Nps_index Nifgs Nest]);
                xcheck(:,:,1) = psp_acheck_grid(ps_index,:,1);
                for w = 2:Nest
                  xcheck(:,:,w) = psp_acheck_grid(ps_index,:,w)
+repmat(psc_acheck_new(ref(w),:)-psc_acheck_new(ref(1),:),Nps_index,1);
                end

                switch dens_check
                  case {'2of3','3of3'}
                    diff12 = sum(abs(xcheck(:,:,2)-xcheck(:,:,1)),2);
                    diff23 = sum(abs(xcheck(:,:,3)-xcheck(:,:,2)),2);
                    diff31 = sum(abs(xcheck(:,:,1)-xcheck(:,:,3)),2);
                end

                switch dens_check
                  case '2of3'

                    %%Two of three correct:
                    for h = 1:Nps_index
                      error_flag = 0;
                      if diff12(h)~=0
                        error_flag = error_flag+1;
                      end
                      if diff23(h)~=0
                        error_flag = error_flag+2;
                      end
                      if diff31(h)~=0
                        error_flag = error_flag+4;
                      end

                      if error_flag==0 % do nothing
                      elseif error_flag==3
                        psp_acheck_grid(ps_index(h),:,2) =
psp_acheck_grid(ps_index(h),:,1)+(psc_acheck_new(ref(1),:)-
psc_acheck_new(ref(2),:));
                        psp_param_grid(ps_index(h),:,2) = -9999;
                        psp_ens_coh_local(ps_index(h),2) = -9999;
                      elseif error_flag==5
                        psp_acheck_grid(ps_index(h),:,1) =
psp_acheck_grid(ps_index(h),:,2)+(psc_acheck_new(ref(2),:)-
psc_acheck_new(ref(1),:));
                        psp_param_grid(ps_index(h),:,1) = -9999;
                        psp_ens_coh_local(ps_index(h),1) = -9999;
                      elseif error_flag==6
                        psp_acheck_grid(ps_index(h),:,3) =
psp_acheck_grid(ps_index(h),:,1)+(psc_acheck_new(ref(1),:)-
psc_acheck_new(ref(3),:));
                        psp_param_grid(ps_index(h),:,3) = -9999;
                        psp_ens_coh_local(ps_index(h),3) = -9999;
                      else
                        psp_acheck_grid(ps_index(h),:,:) = repmat(NaN,[1 Nifgs
3]);
                      end
                    end

                    ps_index = find(isnan(psp_acheck_grid(:,1,1)));
```

```
                case '3of3'

                  %%All three correct:
                  diff_tot = diff12+diff23+diff31;
                  ps_index2 = find(diff_tot~=0);
                  psp_acheck_grid(ps_index(ps_index2),:,:) = repmat(NaN,
[length(ps_index2) Nifgs 3]);
                  ps_index = find(isnan(psp_acheck_grid(:,1,1)));

                case 'mode_fix'

                  %%Take mode solution
                  psp_acheck_grid(ps_index,:,:) = repmat(NaN,[length(ps_index)
Nifgs Nest]);%reset
                  [xcheck_mode,Noccur] = mode(xcheck,3);

                  xcheck_mode(Noccur<=floor(Nest/2)) = NaN;
                  %Nest=3->min=2of3, Nest=4->min=3of4,Nest=5->min=3of5, etc

                  %ps_index2 =
find(sum(isnan(xcheck_mode),2)<=floor(0.25*Nifgs));
                  ps_index2 = find(~isnan(sum(xcheck_mode,2)));
                  psp_acheck_grid(ps_index(ps_index2),:,1) =
xcheck_mode(ps_index2,:);

                  ps_index = find(isnan(psp_acheck_grid(:,1,1)));

              end

            end

            if length(ps_index)>Npsp_grid/2
              model_index = model_index+1; %another iteration
            else
              model_index = Nmodels+1; %to exit while loop
            end
          end %while


          ps_index = find(~isnan(psp_acheck_grid(:,1,1)));
          Nps_grid = length(ps_index);

          switch run_mode
            case 'validation'
              psp_param_grid_shift = permute(psp_param_grid,[1 3 2]);
              valid_out = [kron(ones(Nps_grid,1),ref) kron((Nps(z)+1:Nps(z)
+Nps_grid)',ones(Nest,1)) ...

reshape(psp_param_grid_shift(ps_index,:,1)',Nps_grid*Nest,1) ...

1000*reshape(psp_param_grid_shift(ps_index,:,4)',Nps_grid*Nest,1) ...

reshape(psp_ens_coh_local(ps_index,:),Nps_grid*Nest,1)];

              fwrite(ps_rel_valid_out_fid,valid_out','double');
          end

          ps_az = psp_az_grid(ps_index);
          ps_r = psp_r_grid(ps_index);
          psp_acheck_grid = psp_acheck_grid(ps_index,:,:);
          %psp_ens_coh_local = mean(psp_ens_coh_local(ps_index,:),2);
          psp_ens_coh_local = max(psp_ens_coh_local(ps_index,:),[],2);
          ps_atmo_grid = psp_atmo_grid(ps_index,:);
```

```
          ps_atmo_master_grid = psp_atmo_master_grid(ps_index);
          ps_phase_unw =
2*pi*(psp_acheck_grid(:,:,1)+repmat(psc_acheck_new(ref(1),:),Nps_grid,1))
+psp_phase(psp_index(ps_index),:)-repmat(psc_phase_new(ref_index,:),Nps_grid,1);
          ps_phase_grid = psp_phase_grid(ps_index,:);
          ps_h2ph_grid = psp_h2ph_grid(ps_index,:);
          ps_amp_disp_grid = psp_amp_disp_grid(ps_index);
          ps_amp_grid = psp_amp_grid(ps_index,:);

          switch defo_model_flag
            case 'yes'
              ps_defo_model_grid = psp_defo_model_grid(ps_index,:);
              ps_phase_unw_tot = ps_phase_unw+ps_defo_model_grid;
            otherwise
              ps_phase_unw_tot = ps_phase_unw;
          end


          % ------------------------------------------------------------------
          % Calculation of the parameters of interest
          % ------------------------------------------------------------------
          % Here we can introduce a stochastic model, assuming that the
          % ambiguities are deterministic, hence, a success rate of 1.

          ps_param = repmat(NaN,Nps_grid,Npar_max);
          ps_covar = repmat(NaN,Nps_grid,Npar_max*(Npar_max+1)/2);
          ps_ens_coh = repmat(NaN,Nps_grid,1);
          ps_defo_tot = repmat(NaN,Nps_grid,Nifgs);
          %ps_acheck = repmat(NaN,Nps_grid,Nifgs);
          ps_sig2hat = repmat(NaN,Nps_grid,1);
          ps_phase_res =  repmat(NaN,Nps_grid,Nifgs);

          h2ph = ((mean(psp_dh2ph_grid,1)+psc_h2ph_new(ref_index,:))/2)';
          althyp_index = final_althyp_index;
          [B1Qy2,par_index,covar_index,defo_index] =
ps_model_definitions('model',final_model,Nifgs,h2ph,Btemp,Bdop,std_param);
          B1 = B1Qy2(1:Nifgs,:);

          switch weighting
            case 'unw'
              N = B1'*B1;
              R = chol(N);
              rhs =  R\(R'\B1');
            case {'weight','vce'}
              N = B1'*invQy*B1;
              R = chol(N);
              rhs =  R\(R'\(B1'*invQy));
          end

          for h = 1:Nps_grid

            y = ps_phase_unw(h,:)';
            xhat = rhs*y;
            yhat = B1*xhat;
            ehat = y-yhat;

            y2 = ps_phase_unw_tot(h,:)';
            xhat2 = rhs*y2;
            yhat2 = B1*xhat2;
            ehat2 = y2-yhat2;

            switch weighting
              case 'vce'
```

```
      %% //gkstart
      %% ----------------------------
      %% VCE
      %% ----------------------------
      %VCE=vcem3(A,y,Q_cofactor);
      %Qy_new=VCE(1)*Q_cofactor(:,:,1)+VCE(2)*Q_cofactor(:,:,2);
      %
      %% check for positive definiteness
      %if and(sum(find(eig(Qy_new)<=0))==0, sum(find(VCE<=0))==0)
      %  invQy = inv(Qy_new); % positive definite
      %else
      %  % not positive definite; calculate 1 variance factor instead
      %  Qy_new=((ehat'*invQy*ehat)/b)*sum(Q_cofactor,3); % sig2hat*Qy
      %  invQy = inv(Qy_new);
      %end
      %% //gkend

      Tb = ehat'*invQy*ehat; % overall model test
      sig2hat = Tb/redun;        % estimate variance factor

      Qy_new=sig2hat*Qy;
      invQy_new = inv(Qy_new);

      N = B1'*invQy_new*B1;
      R = chol(N);
      rhs = R\(R'\(B1'*invQy_new));
      xhat = rhs*y;
      yhat = B1*xhat;
      ehat = y-yhat;
      Rinv  = R\eye(size(B1,2));
      Qahat = Rinv*Rinv';

      covar_reshape = reshape(triu(Qahat),1,Npar^2);
      ps_covar(h,covar_index) = covar_reshape(covar_reshape~=0);
      ps_sig2hat(h) = sig2hat;

  end

  ps_param(h,par_index) = xhat2';
  ps_ens_coh(h) = abs((1/Nifgs)*sum(exp(i*ehat')));
  %ps_acheck(h,:) = psp_acheck_grid(h,:,1)+psc_acheck_new(ref(1),:);
  ps_defo_tot(h,:) = (B1(:,defo_index)*xhat2(defo_index)+ehat2)'/m2ph;
  ps_phase_res(h,:) = ehat2';

end



% ------------------------------------------------------------
% Write results to file
% ------------------------------------------------------------

comment_line = repmat(2,Nps_grid,1);

fwrite(ps_fid,[comment_line ps_az ps_r ps_param ps_ens_coh ...
               psp_ens_coh_local ps_sig2hat ps_phase_unw_tot ...
               ps_defo_tot ps_atmo_grid]','double');

switch weighting
  case 'vce'
    fwrite(ps_covar_fid,ps_covar','double');
end

switch run_mode
```

```
            case 'validation'

              if Nps_grid>0

                %create atmo per slc
                %atmo_slave = atmo_master - atmo_ifgs
                ps_atmo_slc = [ps_atmo_grid(:,1:master_index-1)
zeros(Nps_grid,1) ...
                               ps_atmo_grid(:,master_index:end)];
                ps_atmo_slc = repmat(ps_atmo_master_grid,1,Nifgs+1)- ...
                    ps_atmo_slc;

                %create ts
                ps_ts = m2ph*[ps_defo_tot(:,1:master_index-1)
zeros(Nps_grid,1) ...
                              ps_defo_tot(:,master_index:end)];
                %write data
                ps_valid_out = [(Nps(z)+1:Nps(z)+Nps_grid)' ps_az-1 ps_r-1 ...
                                ps_amp_disp_grid ps_amp_grid ps_phase_grid ...
                                m2ph*ps_h2ph_grid ps_param(:,1)
1000*ps_param(:,4) ...
                                ps_ens_coh ps_atmo_grid ps_atmo_slc ps_ts ...
                                ps_phase_res];

                fwrite(ps_valid_out_fid,ps_valid_out','double');

              end

            end

            Nps(z) = Nps(z)+Nps_grid;
          end %for grids
        end %for buffers

        fclose('all');


      case 'whole'

        amp_disp_fid = fopen([project_id '_amp_disp.raw'],'r');
        amp_disp = fread(amp_disp_fid,[Npixels Nlines],'*single')';
        fclose(amp_disp_fid);

        % remove side lobes
        side_lobe_fid = fopen([project_id '_side_lobe_mask.raw'],'r');
        sl_mask = fread(side_lobe_fid,[Npixels Nlines],'*uint8')';
        fclose(side_lobe_fid);

        amp_disp(sl_mask) = NaN; % amp_disp is used for
                                 % administration, also in case of an
                                 % area of interest

        % remove water
        if ischar(filename_water_mask)
          Nlines_file = crop_in(Nifgs+1,2)-crop_in(Nifgs+1,1)+1;
          loffset = crop(1)-crop_in(Nifgs+1,1);
          poffset = crop(3)-crop_in(Nifgs+1,3);
          water_mask = uint8(freadbk(filename_water_mask,Nlines_file,'uint8',...
                        1+loffset,1+loffset+Nlines,...
                        1+poffset,1+poffset+Npixels));

          amp_disp(water_mask) = NaN;
        end
```

```
      % ------------------------------------------------------------
      % Remove PSC's
      % ------------------------------------------------------------

      for v = 1:length(psc)
        amp_disp(psc_az_new(v),psc_r_new(v)) = NaN;
      end

      amp_disp_sort = sort(amp_disp(:));
      Namp_disp_samples = round((Nlines*Npixels/2)/Namp_disp_bins);
      amp_disp_bins =
amp_disp_sort(Namp_disp_samples:Namp_disp_samples:Namp_disp_bins*Namp_disp_sampl
es);

      Ndens_iterations = min(max(1,Ndens_iterations),Namp_disp_bins);
      clear amp_disp_sort

      Nbuffers_az = size(unique(grid_array_az(:,3)),1);
      Nbuffers_r = size(unique(grid_array_r(:,3)),1);

      for v = 1:Nbuffers_az

        index_az = find(grid_array_az(:,3)==v);
        Ngrid_az = length(index_az);
        begin_buffer_az = grid_array_az(index_az(1),1);
        end_buffer_az = grid_array_az(index_az(end),2);
        buffer_size_az = end_buffer_az-begin_buffer_az+1;

        for w = 1:Nbuffers_r

          index_r = find(grid_array_r(:,3)==w);
          Ngrid_r = length(index_r);
          begin_buffer_r = grid_array_r(index_r(1),1);
          end_buffer_r = grid_array_r(index_r(end),2);
          buffer_size_r = end_buffer_r-begin_buffer_r+1;

          ifgs_array = repmat(NaN,[buffer_size_az buffer_size_r Nifgs]);
          atmo_array = repmat(NaN,[buffer_size_az buffer_size_r Nifgs]);
          h2ph_array = repmat(NaN,[buffer_size_az buffer_size_r Nifgs]);

          for k = 1:Nifgs
            Nlines_file = crop_in(k,2)-crop_in(k,1)+1;
            loffset = crop(1)-crop_in(k,1);
            poffset = crop(3)-crop_in(k,3);
            atmo_array(:,:,k) = freadbk([char(filenames_output(k))
'_atmo_sel' ...
                  num2str(z) '.raw'],Nlines_file,'float32',...
                  begin_buffer_az+loffset,end_buffer_az+loffset,...
                  begin_buffer_r+poffset,end_buffer_r+poffset);
            ifgs_array(:,:,k) =
angle(freadbk(char(filenames_ifgs(k)),Nlines_file,'cpxfloat32',...
                  begin_buffer_az+loffset,end_buffer_az+loffset,...
                  begin_buffer_r+poffset,end_buffer_r+poffset));
            h2ph_array(:,:,k) =
freadbk(char(filenames_h2ph(k)),Nlines_file,'float32',...
                  begin_buffer_az+loffset,end_buffer_az+loffset,...
                  begin_buffer_r+poffset,end_buffer_r+poffset);
          end

          ifgs_array = ifgs_array-atmo_array;

          switch detrend_flag
            case 'yes'
```

```
              trend_array = repmat(NaN,[buffer_size_az buffer_size_r Nifgs]);
              for k = 1:Nifgs
                trend_array(:,:,k) = freadbk([char(filenames_output(k))
'_trend_sel' num2str(z)
'.raw'],Nlines,'float32',begin_buffer_az,end_buffer_az,begin_buffer_r,end_buffer
_r);
              end
              ifgs_array = ifgs_array-trend_array;
           end

           switch defo_model_flag
             case 'yes'
              defo_array = repmat(NaN,[buffer_size_az buffer_size_r Nifgs]);
              for k = 1:Nifgs
                defo_array(:,:,k) = freadbk([char(filenames_output(k))
'_defo_sel' num2str(z)
'.raw'],Nlines,'float32',begin_buffer_az,end_buffer_az,begin_buffer_r,end_buffer
_r);
              end
              ifgs_array = ifgs_array-defo_array;
           end

           ifgs_array = mod(ifgs_array+pi,2*pi)-pi;

           if ischar(ps_aoi)
             Nlines_file = crop_in(Nslc,2)-crop_in(Nslc,1)+1;
             loffset = crop(1)-crop_in(Nslc,1);
             poffset = crop(3)-crop_in(Nslc,3);
             aoi_mask = freadbk(ps_aoi,Nlines_file,'uint8',...
                          begin_buffer_az+loffset,end_buffer_az+loffset,...
                          begin_buffer_r+poffset,end_buffer_r+poffset);
           end

           for g = 1:Ngrid_az
             begin_grid_az = grid_array_az(index_az(g),1)-begin_buffer_az+1;
             end_grid_az = grid_array_az(index_az(g),2)-begin_buffer_az+1;
             Nlines_grid = end_grid_az-begin_grid_az+1;

             for h = 1:Ngrid_r
               begin_grid_r = grid_array_r(index_r(h),1)-begin_buffer_r+1;
               end_grid_r = grid_array_r(index_r(h),2)-begin_buffer_r+1;
               Npixels_grid = end_grid_r-begin_grid_r+1;
               Nps_grid_max = Nlines_grid*Npixels_grid;

               psp_atmo_grid =
reshape(atmo_array(begin_grid_az:end_grid_az,begin_grid_r:end_grid_r,:),Nlines_g
rid*Npixels_grid,Nifgs);
               psp_phase_grid =
reshape(ifgs_array(begin_grid_az:end_grid_az,begin_grid_r:end_grid_r,:),Nlines_g
rid*Npixels_grid,Nifgs);
               psp_h2ph_grid =
reshape(h2ph_array(begin_grid_az:end_grid_az,begin_grid_r:end_grid_r,:),Nlines_g
rid*Npixels_grid,Nifgs);
               amp_disp_grid =
reshape(amp_disp(grid_array_az(index_az(g),1):grid_array_az(index_az(g),2),grid_
array_r(index_r(h),1):grid_array_r(index_r(h),2)),Nlines_grid*Npixels_grid,1);

               switch defo_model_flag
                 case 'yes'
                   psp_defo_model_grid =
reshape(defo_array(begin_grid_az:end_grid_az,begin_grid_r:end_grid_r,:),Nlines_g
rid*Npixels_grid,Nifgs);
               end
```

```
              ps_phase_unw_grid = repmat(NaN,Nps_grid_max,Nifgs);
              ps_acheck_grid = repmat(NaN,Nps_grid_max,Nifgs);
              ps_atmo_grid = repmat(NaN,Nps_grid_max,Nifgs);
              ps_az_grid = repmat(NaN,Nps_grid_max,1);
              ps_r_grid = repmat(NaN,Nps_grid_max,1);
              ps_ens_coh_local_grid = repmat(NaN,Nps_grid_max,1);
              ps_comment = repmat(NaN,Nps_grid_max,1);

              switch defo_model_flag
                case 'yes'
                  ps_defo_model_grid = repmat(NaN,Nps_grid_max,Nifgs);
              end

              % ----------------------------------------------------
              % Determine reference points for gridcell
              % ----------------------------------------------------

              center_az = floor((grid_array_az(index_az(g),1) +
grid_array_az(index_az(g),2))/2);
              center_r = floor((grid_array_r(index_r(h),1) +
grid_array_r(index_r(h),2))/2);
              distance = sqrt(((psc_az_new-center_az)*az_spacing).^2 +
((psc_r_new-center_r)*r_spacing).^2);
              [dummy,ref] = sort(distance);
              ref = ref(1:3);

              index = find(amp_disp_grid<amp_disp_bins(1));

              if ~isempty(index)
                Npsp_grid = length(index);
                [az_sub,r_sub] = ind2sub([Nlines_grid Npixels_grid],index);
                psp_az_grid = begin_buffer_az+begin_grid_az+az_sub-2;
                psp_r_grid = begin_buffer_r+begin_grid_r+r_sub-2;

                psp_param_grid = repmat(NaN,[Npsp_grid Npar_max 3]);
                psp_ens_coh_local = repmat(NaN,Npsp_grid,3);
                psp_acheck_grid = repmat(NaN,[Npsp_grid Nifgs 3]);

                for k = 1:3
                  psp_dh2ph_grid = (psp_h2ph_grid(index,:) +
repmat(psc_h2ph_new(ref(k),:),Npsp_grid,1))./2; % take mean for arc
                  psp_dphase_grid = psp_phase_grid(index,:) -
repmat(psc_phase_new(ref(k),:),Npsp_grid,1);


                  % ----------------------------------------------
                  % Estimate the parameters
                  % ----------------------------------------------

                  switch ps_method
                    case 'perio'


[psp_param_grid(:,:,k),psp_acheck_grid(:,:,k),psp_model_info,psp_ens_coh_local(:
,k)] =
ps_periodogram_estimation(psp_dh2ph_grid,Btemp,psp_dphase_grid,ps_model,std_para
m,step1_orig,step2_orig);

                    case {'ils','boot'}


[psp_param_grid(:,:,k),psp_acheck_grid(:,:,k),psp_model_info,psp_varfac_best,psp
_ens_coh_local(:,k)] =
ps_ils_and_bootstrap_estimation(psp_dh2ph_grid,Btemp,Bdop,psp_dphase_grid,ps_mod
```

```
el,std_param,sig2_est,breakpoint,breakpoint2);

                  end
                end

                xcheck = repmat(NaN,[Npsp_grid Nifgs 3]);
                xcheck(:,:,1) = psp_acheck_grid(:,:,1);
                xcheck(:,:,2) =
psp_acheck_grid(:,:,2)+repmat(psc_acheck_new(ref(2),:)-
psc_acheck_new(ref(1),:),Npsp_grid,1);
                xcheck(:,:,3) =
psp_acheck_grid(:,:,3)+repmat(psc_acheck_new(ref(3),:)-
psc_acheck_new(ref(1),:),Npsp_grid,1);
                diff12 = sum(abs(xcheck(:,:,2)-xcheck(:,:,1)),2);
                diff23 = sum(abs(xcheck(:,:,3)-xcheck(:,:,2)),2);
                diff31 = sum(abs(xcheck(:,:,1)-xcheck(:,:,3)),2);

                %%All three correct:
                diff_tot = diff12+diff23+diff31;
                ps_index = find(diff_tot~=0);
                psp_acheck_grid(ps_index,:,:) = repmat(NaN,[length(ps_index)
Nifgs 3]);

                ps_index = find(~isnan(psp_acheck_grid(:,1,1)));
                Nps_grid = length(ps_index);
                ps_az_grid(1:Nps_grid) = psp_az_grid(ps_index);
                ps_r_grid(1:Nps_grid) = psp_r_grid(ps_index);
                ps_acheck_grid(1:Nps_grid,:) =
psp_acheck_grid(ps_index,:,1)+repmat(psc_acheck_new(ref(1),:),Nps_grid,1);
                %ps_ens_coh_local_grid(1:Nps_grid) =
mean(psp_ens_coh_local(ps_index,:),2);
                ps_ens_coh_local_grid(1:Nps_grid) =
max(psp_ens_coh_local(ps_index,:),[],2);
                ps_atmo_grid(1:Nps_grid,:) = psp_atmo_grid(index(ps_index),:);
                ps_phase_unw_grid(1:Nps_grid,:) =
2*pi*ps_acheck_grid(1:Nps_grid,:)+psp_phase_grid(index(ps_index),:)-
repmat(psc_phase_new(ref_index,:),Nps_grid,1);
                ps_comment(1:Nps_grid) = 2;

                switch defo_model_flag
                  case 'yes'
                    ps_defo_model_grid(1:Nps_grid,:) =
psp_defo_model_grid(index(ps_index),:);
                end

                amp_disp_grid(index(ps_index)) = NaN;
                ps_index_old = index(ps_index);
              else
                Nps_grid = 0;
                ps_index_old = [];
              end


              % ------------------------------------------------
              % Densification by nearest neighbour connections
              % ------------------------------------------------

              for p = 2:Ndens_iterations
                [v w g h p]
                index = find(amp_disp_grid<amp_disp_bins(p));


                % ------------------------------------------------
                % If needed, insert area of interest
```

```
                % ------------------------------------------------

                if p==Ndens_iterations & ~isempty(ps_aoi)
                  index_unw = find(isnan(amp_disp_grid));
                  if ischar(ps_aoi)
                    aoi_mask_grid =
reshape(aoi_mask(begin_grid_az:end_grid_az,begin_grid_r:end_grid_r),Nlines_grid*
Npixels_grid,1);
                    index_mask = find(aoi_mask_buffer==1);
                  elseif ~isempty(ps_aoi)
                    aoi_mask_grid_az = max(1,ps_aoi(1)-begin_buffer_az-
begin_grid_az+2):min(Nlines_grid,ps_aoi(2)-begin_buffer_az-begin_grid_az+2);
                    aoi_mask_grid_r = max(1,ps_aoi(3)-begin_buffer_r-
begin_grid_r+2):min(Npixels_grid,ps_aoi(4)-begin_buffer_r-begin_grid_r+2);
                    if ~isempty(aoi_mask_grid_az)& ~isempty(aoi_mask_grid_r)
                      Nmask_az = length(aoi_mask_grid_az);
                      Nmask_r = length(aoi_mask_grid_r);
                      [aoi_mask_grid_az,aoi_mask_grid_r] =
meshgrid(aoi_mask_grid_az,aoi_mask_grid_r);

                      aoi_mask_grid_az =
reshape(aoi_mask_grid_az,Nmask_az*Nmask_r,1);
                      aoi_mask_grid_r =
reshape(aoi_mask_grid_r,Nmask_az*Nmask_r,1);
                      index_mask =
sub2ind([Nlines_grid,Npixels_grid],aoi_mask_rid_az,aoi_mask_grid_r);
                    else
                      index_mask = [];
                    end
                  end
                  index_mask = setdiff(index_mask,index_unw);
                  index = union(index,index_mask);
                end

                if ~isempty(index)
                  Npsp_grid = length(index);
                  [az_sub,r_sub] = ind2sub([Nlines_grid Npixels_grid],index);
                  psp_az_grid = begin_buffer_az+begin_grid_az+az_sub-2;
                  psp_r_grid = begin_buffer_r+begin_grid_r+r_sub-2;

                  ps_index = 1:Nps_grid;

                  if ~isempty(ps_index)
                    psp_neighbour_index = repmat(NaN,Npsp_grid,1);
                    for q = 1:Npsp_grid
                      dist = sqrt(((psp_az_grid(q)-
ps_az_grid(ps_index))*az_spacing).^2+((psp_r_grid(q)-
ps_r_grid(ps_index))*r_spacing).^2);
                      [dummy,psp_neighbour_index(q)] = min(dist);
                    end

                    psp_dh2ph_grid = (psp_h2ph_grid(index,:) ...
                                      +
psp_h2ph_grid(ps_index_old(psp_neighbour_index),:))/2;
                    psp_dphase_grid = psp_phase_grid(index,:) ...
                        - psp_phase_grid(ps_index_old(psp_neighbour_index),:);
                    ps_acheck_ref = ps_acheck_grid(psp_neighbour_index,:);
                  else %in case no ps's in grid yet,
                       %use closest psc
                    psp_dh2ph_grid = (psp_h2ph_grid(index,:) ...
                                      +
repmat(psc_h2ph_new(ref(1),:),Npsp_grid,1))./2;
                    psp_dphase_grid = psp_phase_grid(index,:) ...
                        - repmat(psc_phase_new(ref(1),:),Npsp_grid,1);
```

```
                    ps_acheck_ref =
repmat(psc_acheck_new(ref(1),:),Npsp_grid,1);
                  end

                  % ------------------------------------------------
                  % Estimate the parameters
                  % ------------------------------------------------

                  switch ps_method
                    case 'perio'


[psp_param_grid,psp_acheck_grid,psp_model_info,psp_ens_coh_local] =
ps_periodogram_estimation(psp_dh2ph_grid,Btemp,psp_dphase_grid,ps_model,std_para
m,step1_orig,step2_orig);

                    case {'ils','boot'}


[psp_param_grid,psp_acheck_grid,psp_model_info,psp_varfac_best,psp_ens_coh_local
] =
ps_ils_and_bootstrap_estimation(psp_dh2ph_grid,Btemp,Bdop,psp_dphase_grid,ps_mod
el,std_param,sig2_est,breakpoint,breakpoint2);

                  end

                  ps_index_new = find(~isnan(psp_acheck_grid(:,1)));
                  Npsp_grid = length(ps_index_new);
                  ps_az_grid(Nps_grid+1:Nps_grid+Npsp_grid) =
psp_az_grid(ps_index_new);
                  ps_r_grid(Nps_grid+1:Nps_grid+Npsp_grid) =
psp_r_grid(ps_index_new);
                  %ps_ens_coh_local_grid(Nps_grid+1:Nps_grid+Npsp_grid) =
mean(psp_ens_coh_local(ps_index_new),2);
                  ps_ens_coh_local_grid(Nps_grid+1:Nps_grid+Npsp_grid) =
max(psp_ens_coh_local(ps_index_new),[],2);
                  ps_atmo_grid(Nps_grid+1:Nps_grid+Npsp_grid,:) =
psp_atmo_grid(index(ps_index_new),:);
                  ps_acheck_grid(Nps_grid+1:Nps_grid+Npsp_grid,:) =
psp_acheck_grid(ps_index_new,:)+ps_acheck_ref(ps_index_new,:);
                  ps_phase_unw_grid(Nps_grid+1:Nps_grid+Npsp_grid,:) =
2*pi*ps_acheck_grid(Nps_grid+1:Nps_grid+Npsp_grid,:)
+psp_phase_grid(index(ps_index_new),:)-
repmat(psc_phase_new(ref_index,:),Npsp_grid,1);
                  ps_comment(Nps_grid+1:Nps_grid+Npsp_grid) = p+1;

                  switch defo_model_flag
                    case 'yes'
                      ps_defo_model_grid(Nps_grid+1:Nps_grid+Npsp_grid,:) =
psp_defo_model_grid(index(ps_index_new),:);
                  end

                  amp_disp_grid(index(ps_index_new)) = NaN;
                  Nps_grid = Nps_grid+Npsp_grid;
                  ps_index_old = [ps_index_old;index(ps_index_new)];
                end
              end


              % ------------------------------------------------
              % Remove non-unwrapped pixels
              % ------------------------------------------------

              index = find(isnan(ps_az_grid));
```

```
                ps_az_grid(index) = [];
                ps_r_grid(index) = [];
                ps_ens_coh_local_grid(index) = [];
                ps_atmo_grid(index,:) = [];
                ps_phase_unw_grid(index,:) = [];
                ps_comment(index) = [];

                switch defo_model_flag
                  case 'yes'
                    ps_defo_model_grid(index,:) = [];
                end

                h2ph = mean(psp_h2ph_grid,1);


                switch densification_flag
                  case 'yes'

                    % ------------------------------------------------
                    % Densification by growing region
                    % ------------------------------------------------

                    ps_phase_array =
mod(ifgs_array(begin_grid_az:end_grid_az,begin_grid_r:end_grid_r,:)-
repmat(reshape(psc_phase_new(ref_index,:),[1 1 Nifgs]),[Nlines_grid Npixels_grid
1])+pi,2*pi)-pi;
                    ps_ens_coh_array = repmat(NaN,Nlines_grid,Npixels_grid);
                    ps_flags_array = zeros(Nlines_grid,Npixels_grid,'int8');
                    az_sub_orig = ps_az_grid-begin_buffer_az-begin_grid_az+2;
                    r_sub_orig = ps_r_grid-begin_buffer_r-begin_grid_r+2;
                    index_orig = sub2ind([Nlines_grid
Npixels_grid],az_sub_orig,r_sub_orig);
                    ps_comment_orig = ps_comment;

                    for k = 1:Nps_grid
                      ps_phase_array(ps_az_grid(k)-begin_buffer_az-
begin_grid_az+2,ps_r_grid(k)-begin_buffer_r-begin_grid_r+2,:) =
ps_phase_unw_grid(k,:);
                      ps_ens_coh_array(ps_az_grid(k)-begin_buffer_az-
begin_grid_az+2,ps_r_grid(k)-begin_buffer_r-begin_grid_r+2) =
ps_ens_coh_local_grid(k);
                      ps_flags_array(ps_az_grid(k)-begin_buffer_az-
begin_grid_az+2,ps_r_grid(k)-begin_buffer_r-begin_grid_r+2) = 1;
                    end

                    start_list = [ps_az_grid-begin_buffer_az-begin_grid_az+2
ps_r_grid-begin_buffer_r-begin_grid_r+2 ps_ens_coh_local_grid];

                    [ps_phase_array,ps_ens_coh_array,ps_flags_array] =
ps_region_growing(ps_phase_array,ps_ens_coh_array,ps_flags_array,start_list,Btem
p,Bdop,std_param,h2ph,final_model,final_althyp_index);


                    % ------------------------------------------------
                    % Construct new output
                    % ------------------------------------------------

                    [az_sub,r_sub] = find(ps_flags_array==1);
                    ps_az_grid = az_sub+begin_buffer_az+begin_grid_az-2;
                    ps_r_grid = r_sub+begin_buffer_r+begin_grid_r-2;
                    Nps_grid = length(ps_az_grid);
                    index = sub2ind([Nlines_grid Npixels_grid],az_sub,r_sub);

                    ps_phase_unw_grid =
```

```
reshape(ps_phase_array,Nlines_grid*Npixels_grid,Nifgs);
                ps_ens_coh_local_grid =
reshape(ps_ens_coh_array,Nlines_grid*Npixels_grid,1);
                ps_comment = repmat(NaN,Nlines_grid*Npixels_grid,1);
                ps_comment(index) = -1;
                ps_comment(index_orig) = ps_comment_orig;


                % ------------------------------------------------
                % If needed, remove psc
                % ------------------------------------------------

                index_psc = find(ismember([ps_az_grid ps_r_grid],[psc_az
psc_r],'rows'));
                if ~isempty(index_psc)
                  index(index_psc) = [];
                  ps_az_grid(index_psc) = [];
                  ps_r_grid(index_psc) = [];
                  Nps_grid = length(index);
                end

                ps_phase_unw_grid = ps_phase_unw_grid(index,:);
                ps_atmo_grid = psp_atmo_grid(index,:);
                ps_ens_coh_local_grid = ps_ens_coh_local_grid(index);
                ps_comment = ps_comment(index);

                switch defo_model_flag
                  case 'yes'
                    ps_defo_model_grid = psp_defo_model_grid(index,:);
                end

            end


            switch defo_model_flag
              case 'yes'
                ps_phase_unw_grid_tot = ps_phase_unw_grid+ps_defo_model_grid;
              otherwise
                ps_phase_unw_grid_tot = ps_phase_unw_grid;
            end


            % ------------------------------------------------
            % Calculation of the parameters of interest
            % ------------------------------------------------
            % Here we can introduce a stochastic model, assuming that the
            % ambiguities are deterministic, hence, a success rate of 1.

            ps_param_grid = repmat(NaN,Nps_grid,Npar_max);
            ps_covar_grid = repmat(NaN,Nps_grid,Npar_max*(Npar_max+1)/2);
            ps_ens_coh_grid = repmat(NaN,Nps_grid,1);
            ps_defo_tot = repmat(NaN,Nps_grid,Nifgs);
            ps_sig2hat_grid = repmat(NaN,Nps_grid,1);

            h2ph = ((mean(psp_dh2ph_grid,1)+psc_h2ph_new(ref_index,:))/2)';
            % This is not completely right, but should
            % approximately be oke
            althyp_index = final_althyp_index;
            [B1Qy2,par_index,covar_index,defo_index] =
ps_model_definitions('model',final_model,Nifgs,h2ph,Btemp,Bdop,std_param);
            B1 = B1Qy2(1:Nifgs,:);

            switch weighting
              case 'unw'
```

```
                N = B1'*B1;
                R = chol(N);
                rhs =  R\(R'\B1');
              case {'weight','vce'}
                N = B1'*invQy*B1;
                R = chol(N);
                rhs =  R\(R'\(B1'*invQy));
            end

            for k = 1:Nps_grid

              y = ps_phase_unw_grid(k,:)';
              xhat = rhs*y;
              yhat = B1*xhat;
              ehat = y-yhat;

              y2 = ps_phase_unw_grid_tot(k,:)';
              xhat2 = rhs*y2;
              yhat2 = B1*xhat2;
              ehat2 = y2-yhat2;

              switch weighting
                case 'vce'

                  Tb = ehat'*invQy*ehat; % overall model test
                  sig2hat = Tb/redun;        % estimate variance factor

                  Qy_new=sig2hat*Qy;
                  invQy_new = inv(Qy_new);

                  N = B1'*invQy_new*B1;
                  R = chol(N);
                  rhs = R\(R'\(B1'*invQy_new));
                  xhat = rhs*y;
                  yhat = B1*xhat;
                  ehat = y-yhat;
                  Rinv  = R\eye(size(B1,2));
                  Qahat = Rinv*Rinv';

                  covar_reshape = reshape(triu(Qahat),1,Npar^2);
                  ps_covar_grid(k,covar_index) =
covar_reshape(covar_reshape~=0);
                  ps_sig2hat_grid(k) = sig2hat;

              end

              ps_param_grid(k,par_index) = xhat2';
              ps_ens_coh_grid(k) = abs((1/Nifgs)*sum(exp(i*ehat')));
              ps_defo_tot(k,:) = (B1(:,defo_index)*xhat2(defo_index)
+ehat2)'/m2ph;

            end



            % ----------------------------------------------------------------
            % Write results to file
            % ----------------------------------------------------------------

            fwrite(ps_fid,[ps_comment ps_az_grid ps_r_grid ps_param_grid ...
                           ps_ens_coh_grid ps_ens_coh_local_grid ...
                           ps_sig2hat_grid ps_phase_unw_grid_tot ...
                           ps_defo_tot ps_atmo_grid]','double');
```

```
              switch weighting
                case 'vce'
                  fwrite(ps_covar_fid,ps_covar_grid','double');
              end

              Nps(z) = Nps(z)+Nps_grid;

            end
          end
        end
      end


    otherwise

      error('You specified a wrong ps_eval_method.');

  end

end
```